FastAPI → Agentic AI

Complete Learning Roadmap

Structure • Order • Projects • No Shortcuts

Sahi Order — Kyun Important Hai

RAG ek backend application hai. Bina backend structure ke RAG banana = 300-line spaghetti file jo production mein crash karegi. Pehle foundation, phir AI.

Step	Kya Sikhna Hai	Kyun Pehle
1	Python Basics + async/await	Har cheez iske upar build hoti hai
2	FastAPI Structure (Blog API)	RAG ka backbone yehi hai
3	LLM Raw Calls	Wrappers samajhne se pehle internals
4	RAG System	Ab structure pata hai — sahi jagah sab jayega
5	AI Agents (LangGraph)	FastAPI ke andar — replace nahi karta

FastAPI Project Structure — Universal

Ye structure har project mein same rehta hai — Blog API ho, RAG ho, ya Agent backend. Sirf files change hoti hain, skeleton nahi.

```
project/
├── app/
│   ├── main.py            # App init, lifespan, middleware register
│   ├── config.py          # Settings via pydantic-settings (.env)
│   ├── dependencies.py    # Depends() — auth, db session, rate limit
│   │
│   ├── routers/           # HTTP layer only — thin, no logic
│   │   ├── users.py
│   │   └── posts.py
│   │
│   ├── models/            # SQLAlchemy ORM — DB table shape
│   │   └── user.py
│   │
│   ├── schemas/           # Pydantic — request/response shape
│   │   └── user.py        # UserCreate vs UserResponse — ALAG rakhna
│   │
│   ├── services/          # Business logic — ≈ controller
│   │   └── user_service.py
│   │
│   ├── repositories/      # Raw DB queries — service se alag
│   │   └── user_repo.py
│   │
│   └── middleware/        # Logging, CORS, custom headers
```

Golden Rule

Router → Service → Repository → DB. Kabhi ulta nahi. Router mein DB query nahi. Repository mein business logic nahi. Ye rule tod ke commit mat karna.

Har Layer Kya Karta Hai

routers/	Sirf HTTP — validate karo, service call karo, return karo. 10-15 lines max per endpoint.
schemas/	Pydantic models. UserCreate (input) aur UserResponse (output) ALAG files. Kabhi DB model return mat karo directly — password leak hoga.
services/	Business logic. Agar rule change ho toh sirf yahan change karo. Repository directly call nahi karta — service ke through.
repositories/	Pure DB queries. SQLAlchemy. Koi business logic nahi. Agar query badalni hai toh sirf yahan aao.
models/	SQLAlchemy ORM table definitions. Ye DB ka shape hai — API ka nahi.
dependencies.py	Depends() functions — get_db(), get_current_user(), rate_limiter(). Har route mein inject hote hain.

Node.js vs FastAPI — Key Differences

Concept	Express / Node.js	FastAPI / Python
Validation	joi / zod — manual setup	Pydantic — built-in, auto 422
Auth per route	middleware chaining	Depends(get_current_user)
Response shape	res.json(user) — leaks fields	response_model= — auto filter
DB session	manual per request	Depends(get_db) — auto close
Controller	one layer	services/ + repositories/ (split)
Config	process.env	pydantic-settings — typed
Async	opt-in	async def — native

Step 1 — Blog API (Pehla Project)

Blog API banao — Todo nahi. Blog mein posts, comments, auth, pagination sab aata hai ek project mein. Ye structure haath mein aa jaayega.

```
project/
■■■ app/
■   ■■■ routers/
■   ■   ■■■ users.py      # /users — signup, login
■   ■   ■■■ posts.py      # /posts — CRUD
■   ■   ■■■ comments.py   # /posts/{id}/comments
■   ■
■   ■■■ models/
■   ■   ■■■ user.py       # User table
■   ■   ■■■ post.py       # Post table
■   ■   ■■■ comment.py    # Comment table
■   ■
■   ■■■ schemas/
■   ■   ■■■ user.py       # UserCreate, UserResponse
■   ■   ■■■ post.py       # PostCreate, PostUpdate, PostResponse
■   ■   ■■■ comment.py
■   ■
■   ■■■ services/
■   ■   ■■■ user_service.py
■   ■   ■■■ post_service.py
■   ■   ■■■ comment_service.py
■   ■
■   ■■■ repositories/
■       ■■■ user_repo.py
■       ■■■ post_repo.py
■       ■■■ comment_repo.py
```

Blog API Checklist	
CRUD for posts + comments	✓
JWT auth — signup, login, refresh token	✓
Pydantic validation — proper error messages	✓
Services layer — no logic in routers	✓
Repositories layer — no raw queries in services	✓
Depends() for auth + DB session	✓
pytest — happy path + error path tests	✓
Structured JSON logging	✓
.env secrets — never hardcode	✓

Step 2 — RAG System (Same Structure + Extra)

RAG sirf Blog API ka structure extend karta hai. Core same rehta hai — sirf RAG-specific files add hoti hain aur ek naya pipelines/ folder.

```
project/
■■■ app/
■   ■■■ routers/
■   ■   ■■■ rag.py           # POST /query, POST /ingest  ← ADD
■   ■
```

```
■   ■■■ models/
■   ■   ■■■ document.py       # Document table            ← ADD
■   ■
■   ■■■ schemas/
■   ■   ■■■ rag.py            # QueryRequest, QueryResponse ← ADD
■   ■
■   ■■■ services/
■   ■   ■■■ rag_service.py    # Orchestrate pipeline       ← ADD
■   ■   ■■■ llm_service.py    # OpenAI / Anthropic calls   ← ADD
■   ■   ■■■ embeddings.py     # text → vector               ← ADD
■   ■
■   ■■■ repositories/
■   ■   ■■■ vector_repo.py    # pgvector / Qdrant queries  ← ADD
■   ■   ■■■ document_repo.py  # raw chunks in PostgreSQL   ← ADD
■   ■
■   ■■■ pipelines/            # ← ONLY NEW FOLDER
■   ■   ■■■ ingest.py         # PDF → chunk → embed → store
■   ■   ■■■ retrieve.py       # query → embed → search → rerank
■   ■
■   ■■■ utils/                # ← ADD
■       ■■■ chunking.py       # text todna chunks mein
■       ■■■ pdf_parser.py     # PDF se text nikalna
```

RAG Flow — Ingest (ek baar hota hai)

PDF upload → text extract → chunks banao → har chunk embed karo → vector DB mein store karo → raw text PostgreSQL mein

RAG Flow — Query (har request pe)

User ka question → question embed karo → vector DB mein similar chunks dhundo → rerank karo → LLM ko context do → answer return karo

Pipelines — Import Chain

```python
# pipelines/ingest.py imports
from app.services.embeddings import EmbeddingService
from app.repositories.vector_repo import VectorRepo
from app.repositories.document_repo import DocumentRepo
from app.utils.chunking import chunk_text
from app.utils.pdf_parser import extract_text

# pipelines/retrieve.py imports
from app.services.embeddings import EmbeddingService
from app.repositories.vector_repo import VectorRepo

# services/rag_service.py imports
from app.pipelines.ingest import ingest_document
from app.pipelines.retrieve import retrieve
from app.services.llm_service import LLMService

# routers/rag.py imports
from app.services.rag_service import RAGService
```

Import Rule — Kabhi Mat Todo

router → service → pipeline → repository. Ulta kabhi nahi: repository → service NAHI, pipeline → router NAHI.

Step 3 — Agents (LangGraph Inside FastAPI)

Agents FastAPI replace nahi karte — andar rehte hain. LangGraph ek library hai jo FastAPI ke service layer mein use hoti hai.

```
# services/agent_service.py
from langgraph.graph import StateGraph   # library import
from app.repositories.vector_repo import VectorRepo

class AgentService:
    def __init__(self):
        self.graph = self._build_graph()   # LangGraph setup

    async def run(self, query: str):
        result = await self.graph.ainvoke({"query": query})
        return result

# routers/agent.py — same pattern, kuch naya nahi
@router.post("/agent/run")
async def run_agent(
    query: str,
    agent: AgentService = Depends(),
    user: User = Depends(get_current_user),
):
    return await agent.run(query)
```

Technology	Role	FastAPI Replace Karta?
LangGraph	Agent workflow logic	Nahi — service layer mein
LangChain	Tools, chains, prompts	Nahi — service layer mein
CrewAI / Agno	Multi-agent orchestration	Nahi — service layer mein
RAG Pipeline	Knowledge retrieval	Nahi — pipelines/ mein
pgvector / Qdrant	Vector storage	Nahi — repository layer
Redis	Cache + job queues	Nahi — infrastructure
FastAPI	HTTP layer — sab wrap karta hai	**YE CORE HAI**

Common Mistakes — Jo Mat Karna

✗ Logic router mein likhna	Router mein DB query ya business logic — immediate red flag in code review. Router sirf validate + service call + return karta hai.
✗ models/ aur schemas/ mix karna	SQLAlchemy model (DB shape) aur Pydantic schema (API shape) ALAG hote hain. DB model directly return karo toh password hash leak ho sakta hai.
✗ Ingest endpoint synchronous banana	PDF processing 30 seconds le sakti hai. Synchronous endpoint server block kar dega. BackgroundTasks ya Celery use karo.
✗ Bina metadata ke chunks store karna	Sirf text aur vector store kiya — source file, page number, user_id kuch nahi. Citations nahi de sakte, multi-tenant leakage ka risk hai.
✗ Services layer skip karna	Seedha router se repository call karna. Business logic phir scatter ho jaati hai. Kal requirement change hui — 5 jagah fix karna padega.

Production Stack — Tera Target

Layer	Technology	Kyun
Frontend	React / Next.js	Industry standard
Backend	FastAPI (Python)	AI ecosystem ke saath best fit
Database	PostgreSQL	Production grade, free, reliable
Vector DB	pgvector (PostgreSQL mein)	Alag DB ki zaroorat nahi
Cache + Queue	Redis	Background jobs + caching
Agent Logic	LangGraph	Industry standard 2025-26
LLM Routing	LiteLLM	Multi-provider, cost control
Observability	LangSmith / Arize Phoenix	Traces, evals, debugging
Deployment	Docker + GitHub Actions	CI/CD pipeline

Bottom Line

Blog API → LLM Calls → RAG → Agents. Ye order hai. Skip mat karna.

FastAPI structure universal hai — RAG aur Agents isme fit hote hain, ise replace nahi karte.

Router thin rakho. Services mein logic. Repositories mein queries. Pipelines mein RAG flow.

Projects > Theory. GitHub > Resume. Build > Watch.